

CrypTen 中 LoRA-MPC 训练与性能优化报告

第三周工作汇报

1 本周主要内容

1. 将 LoRA 接入 CrypTen 的 MPC 训练，并保持正确性
2. 尝试调优，减少开销

- LoRA 已通过 MPCLoRALinear + AutogradLoRALinear 在 mpc 中正确实现，模型性能与明文侧 lora 相当。
- 配置各算子迭代轮次，保持正确性的前提下，减少 14% 总时长

2 将 LoRA 集成到 CrypTen MPC 大模型训练

2.1 目标

- 代码正常运行
- 正确性相对 plain-lora 不回退

2.2 当前实现

- LoRA module 定义;

```
SHAFT > build > lib > crypten > nn > module.py > ...
class MPCLoRALinear(Module):
    def __init__(
        self, in_features, out_features, device=device, dtype=dtype,
        requires_grad=True,
    ):
        self.register_parameter(
            "lora_B",
            torch.zeros((out_features, r), device=device, dtype=dtype),
            requires_grad=True,
        )
        torch.nn.init.kaiming_uniform_(self.lora_A, a=5**0.5)

        self.r = r
        self.alpha = float(alpha)
        self.scaling = self.alpha / float(r)
        self.dropout_p = 0.0

    def forward(self, x):
        bias = self.bias if hasattr(self, "bias") else None
        if isinstance(x, crypten.CrypTensor):
            return x.lora_linear(self.weight, self.lora_A, self.lora_B, self.scaling, bias)

        out = x.matmul(self.weight.t())
        if bias is not None:
            out = out.add(bias)
        lora = x.matmul(self.lora_A.t()).matmul(self.lora_B.t())
        return out.add(lora.mul(self.scaling))
```

图 1: LoRA module 定义

- LoRA Op 定义;

```
SHAFT > build > lib > crypten > gradients.py > AutogradLoRALinear > backward
class AutogradLoRALinear(AutogradFunction):
    @staticmethod
    def forward(ctx, input, weight, lora_A, lora_B, scaling, bias=None):
        # LoRA forward:
        # y = x W^T + h B^T * s + b
        # h = x A^T
        non_differentiable = [weight]
        if bias is not None:
            non_differentiable.append(bias)
        ctx.mark_non_differentiable(non_differentiable)
        output = input.matmul(weight.t())
        # 保存 h = x A^T 至 ctx
        lora_hidden = input.matmul(lora_A.t())
        ctx.save_multiple_for_backward([input, weight, lora_A, lora_B, bias, scaling, lora_hidden])
        output = output.add(lora_hidden.matmul(lora_B.t()).mul(scaling))
        if bias is not None:
            output = output.add(bias)
        return output
```

图 2: op-forward 定义

```
SHAFT > build > lib > crypten > gradients.py > AutogradLoRALinear > backward
class AutogradLoRALinear(AutogradFunction):
    def backward(ctx, grad_output):
        input_2d = input.reshape(-1, in_features)
        grad_output_2d = grad_output.reshape(-1, out_features)
        lora_hidden_2d = lora_hidden.reshape(-1, lora_hidden.size(-1))
        # backward
        # dL/dx_base = dL/dy W
        # dL/dh_base = dL/dy B
        # dL/dW = (dL/dy)^T x
        # dL/dB = (dL/dy)^T h
        grouped_specs = []
        grouped_keys = []

        if input_requires_grad:
            grouped_specs.append((weight, False))
            grouped_keys.append("grad_input_base_2d")
        if input_requires_grad or lora_A_requires_grad:
            grouped_specs.append((lora_B, False))
            grouped_keys.append("grad_lora_hidden_base_2d")
        if weight_requires_grad:
            grouped_specs.append((input_2d, True))
            grouped_keys.append("grad_weight")
        if lora_B_requires_grad:
            grouped_specs.append((lora_hidden_2d, True))
            grouped_keys.append("grad_lora_B")

        grouped_results = {}
        if grouped_specs:
            grouped_values = _grouped_shared_left_matmuls(grad_output_2d, grouped_specs)
            grouped_results = dict(zip(grouped_keys, grouped_values))
```

图 3: op-backward 定义

- Grouped Shared MatMul 实现

```

SHAFT > build > lib > crypten > gradients.py > _grouped_shared_left_matmuls

def _grouped_shared_left_matmuls(left, grouped_args):
    """Runs a set of matmuls that may share one left reveal."""
    try:
        from crypten.mpc.mpc import MPCTensor
        from crypten.mpc.primitives import beaver as beaver_protocol
        from crypten.mpc.ptype import ptype as Ptype
    except Exception:
        return [(left.t() if transpose_left else left).matmul(right) for right, transpose_left in grouped_args]

    # 不符合条件则退化为普通 matmul
    if not isinstance(left, MPCTensor) or left.ptype != Ptype.arithmetic or left.dim() != 2:
        return [(left.t() if transpose_left else left).matmul(right) for right, transpose_left in grouped_args]

    for right, _ in grouped_args:
        if not isinstance(right, MPCTensor) or right.ptype != Ptype.arithmetic or right.dim() != 2:
            return [(left.t() if transpose_left else left).matmul(right) for right, transpose_left in grouped_args]
    # 转接到beaver.py
    raw_results = beaver_protocol.shared_left_matmul_group(
        left._tensor,
        [(right._tensor, transpose_left) for right, transpose_left in grouped_args],
    )

```

图 4: 接口

```

b > crypten > mpc > primitives > beaver.py > shared_left_matmul_group

def shared_left_matmul_group(x, grouped_args):

    provider = crypten.mpc.get_default_provider()
    # 左操作数共享，右操作数可以不同
    # 共享一份左侧随机掩码 a
    # 为每个右操作数分别生成 b_i 与 c_i = a * b_i
    if not hasattr(provider, "generate_additive_triples_with_same_left"):
        return [matmul(x.t() if transpose_left else x, y) for y, transpose_left in grouped_args]
    right_specs = []
    for y, transpose_left in grouped_args:
        if x.device != y.device:
            raise ValueError(f"x lives on device {x.device} but y on device {y.device}")
        right_specs.append({"right_size": y.size(), "transpose_left": transpose_left})

    try:
        a, triples = provider.generate_additive_triples_with_same_left(
            x.size(), right_specs, device=x.device
        )
    except NotImplementedError:
        return [matmul(x.t() if transpose_left else x, y) for y, transpose_left in grouped_args]
    bs = [b for b, _ in triples]
    cs = [c for _, c in triples]
    from .arithmetic import ArithmeticSharedTensor
    # 对共享左操作数只 reveal 一次 epsilon = x - a。
    with IgnoreEncodings([a, x]):
        epsilon = ArithmeticSharedTensor.reveal(x - a)
    # 每个右操作数 y_i 单独 reveal delta_i = y_i - b_i。
    with IgnoreEncodings(bs + [y for y, _ in grouped_args]):
        deltas = ArithmeticSharedTensor.reveal_batch([y - b for (y, _), b in zip(grouped_args, bs)])

```

图 5: 实现

```

ib > crypten > mpc > provider > tfp_provider.py > TrustedFirstParty > generate_additive_triples_with_same_left
class TrustedFirstParty(TupleProvider):
    def generate_additive_triples_with_same_left(self, left_size, right_specs, device=None):
        """Generate multiple additive triples that reuse one left mask."""
        # 1. 先生成一份所有分支共享的左侧随机掩码 a, 再为每个右操作数分别生成自己的 b_i
        # 3. 计算 c_i = a * b_i
        a_plain = generate_random_ring_element(left_size, device=device)

        triples = []
        for spec in right_specs:
            # 随机生成b_i并计算c_i = a * b_i
            b_plain = generate_random_ring_element(spec["right_size"], device=device)
            left_plain = a_plain.t() if spec.get("transpose_left", False) else a_plain
            c_plain = torch.matmul(left_plain, b_plain)
            triples.append(
                (
                    ArithmeticSharedTensor(b_plain, precision=0, src=0),
                    ArithmeticSharedTensor(c_plain, precision=0, src=0),
                )
            )
        a_shared = ArithmeticSharedTensor(a_plain, precision=0, src=0)
        return a_shared, triples

```

图 6: beaver tuple 复用生成

2.3 结果验证

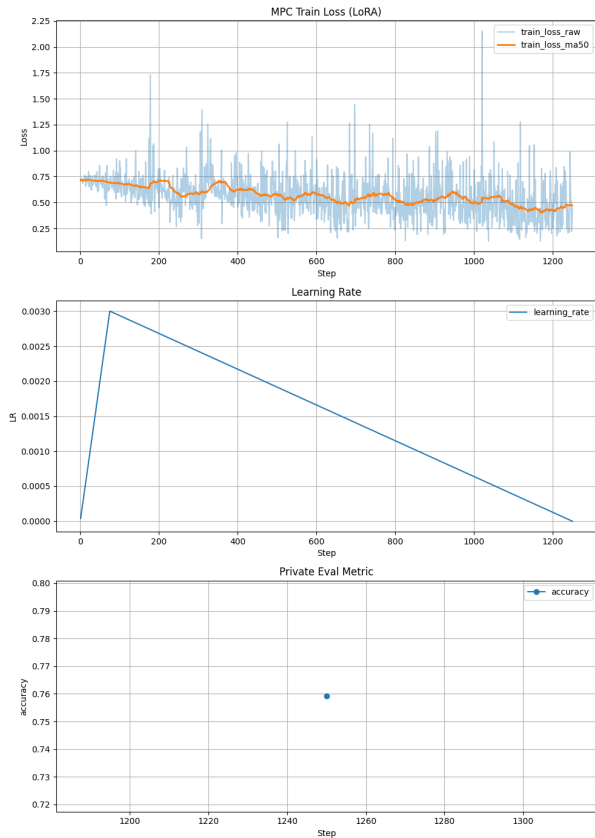


图 7: mpc-lora

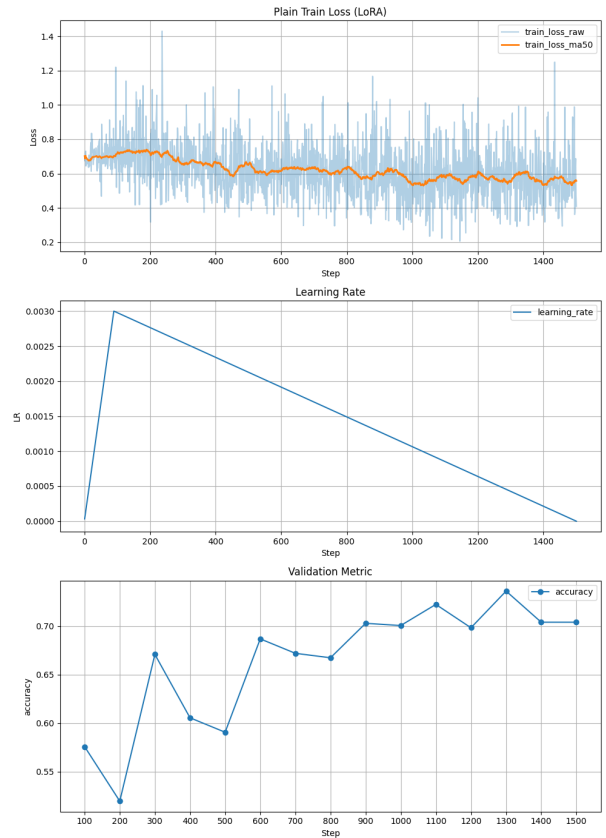


图 8: plain-lora

表 1: LoRA private/plain 对比

配置	last loss	eval loss	acc
plain lora	0.55	0.54	0.73
private lora	0.45	0.49	0.76

3 调优尝试

3.1 目标

- 保持当前训练质量
- 降低单步训练时长

3.2 LoRA backward 侧优化: ctx 修正

前向

$$y = xW^\top + (xA^\top)B^\top \cdot s + b, \quad h = xA^\top,$$

中的中间量 $h = xA^\top$ 在反向中也需要计算

$$\frac{\partial \mathcal{L}}{\partial B} = \left(\frac{\partial \mathcal{L}}{\partial y} \right)^\top h \cdot s$$

因此需要将 $xA^\top$ 放入 ctx。

```
class AutogradLoRALinear(AutogradFunction):
    def forward(ctx, input, weight, lora_A, lora_B, scaling, bias=None):
        output = input.matmul(weight.t())
        # 保存 h = x A^T 至 ctx
        lora_hidden = input.matmul(lora_A.t())
        ctx.save_multiple_for_backward([input, weight, lora_A, lora_B, bias, s
```

图 9: ctx-f

```
class AutogradLoRALinear(AutogradFunction):
    def backward(ctx, grad_output):
        input, weight, lora_A, lora_B, bias, scaling, lora_hidden = ctx.saved_tensors

        input_requires_grad = getattr(input, "requires_grad", False)
        weight_requires_grad = getattr(weight, "requires_grad", False)
        lora_A_requires_grad = getattr(lora_A, "requires_grad", False)
        lora_B_requires_grad = getattr(lora_B, "requires_grad", False)
        bias_requires_grad = bias is not None and getattr(bias, "requires_grad", False)
```

图 10: ctx-b

表 2: ctx 修正前后对比

配置	10-step 总耗时	50-step 总耗时	11-50 warmed-up 单步
1	84.66s	418.24s	8.3395s
2	81.21s	398.33s	约快 4.93%

3.3 softmax / invsqrt 开销优化

- LoRA backward 边际收益较小
- softmax 和 invsqrt 的 `nr_iters` 压缩空间很小

新增了独立开关：

`sqrt_nr_initial_exp_iterations`

它只影响 invsqrt 初值中的 `exp` 函数中的迭代次数

```
ib > crypten > common > functions > approximations.py > inv_sqrt
def inv_sqrt(self):
    if initial is None:
        initial_arg = self.div(2).add(0.2).neg()
        if initial_exp_iters is None:
            y = exp(initial_arg).mul(2.2).add(0.2)
        else:
            with cfg.temp_override({"functions.exp_iterations": initial_exp_iters}):
                y = exp(initial_arg).mul(2.2).add(0.2)
            # y = y - self.div(1024)
            y = y - self.div(linear_divisor)
    else:
        y = initial
```

图 11: exp-iter

短程 50step 下扫描：

- initexp5
- initexp6
- initexp7
- initexp8(default)
- softmax16(default)
- softmax15
- softmax14
- softmax13
- invsqrt10
- invsqrt9
- invsqrt8
- invsqrt7

最终选中（保守地）：initexp7/softmax15/invsqrt10

3.4 长程验证

表 3: baseline 与 iter7/15/10 的长程对比

配置	final accuracy	final eval loss	elapsed time
plain	0.73	/	/
lora-baseline	0.7615	0.4845063	7575.69s
iter7/15/10	0.7661	0.4846269	6529.98s

- acc 提升约 0.46 个百分点
- eval loss 基本持平
- 总耗时减少约 1045.7s, 缩短约 13.8%

4 长程对照试验的共享训练设置

- task: SST-2
- model: prajjwal1/bert-tiny
- train steps: 1250
- train / eval size: 10000 / 872
- shuffle: False
- train / eval batch size: 8 / 8
- optimizer: SGD
- lr: 3×10^{-3}
- momentum: 0.9
- weight decay: 0
- warmup steps / ratio: 75 / 0.06
- loss: manual one-hot cross-entropy
- private loss mode: manual
- trainable scope: LoRA
- LoRA: $r = 8$, $\alpha = 16$, dropout = 0
- target modules: query, value
- replaced modules: 4
- layers:
 $\{\text{layer0.query, layer0.value, layer1.query, layer1.value}\}$

5 LoRA-MPC 训练阶段相关记录文件地址

公共前缀 `examples/text-classification/`。包含 `sst2/{summary.json, train_history.json, eval_history.json, training_curves.png}`。

5.1 初始 LoRA-MPC 训练与 Probe

- `eval_private_control_bert_tiny_lora_sgd_qv_step10_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step1_ode_ceideal_nr10div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step10_ode_ceideal_nr10div1536_qv_noeval_probe`

5.2 Softmax / InvSqrt 主迭代组合扫描

- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr10div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode14_ceideal_nr10div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode15_ceideal_nr10div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr9div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode16_nr10_reusehidden`
- `eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode14_nr10_reusehidden`
- `eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode15_nr10_reusehidden`
- `eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode16_nr9_reusehidden`
- `eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode14_nr9_reusehidden`

5.3 Single-Ideal 对照与 1250 Step Ideal 对照

- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ideal_ceideal_nr10div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_ideal_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_e872_step1250_ideal_qv`

5.4 InitExp 局部 Exp 迭代扫描

- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr10div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr10_initexp5_div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr10_initexp6_div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr10_initexp6_div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr10_initexp7_div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_t10000_step50_ode16_ceideal_nr10_initexp7_div1536_qv_noeval_probe`
- `eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode16_nr10_reusehidden`

- eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode16_nr10_initexp5_reusehidden
- eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode16_nr10_initexp6_reusehidden
- eval_private_control_bert_tiny_lora_sgd_tiny8_step4_ode16_nr10_initexp7_reusehidden
- eval_private_control_bert_tiny_lora_sgd_t10000_e872_step1250_ode_ceideal_nr10div1536_qv
- eval_private_control_bert_tiny_lora_sgd_t10000_e872_step1250_ode_ceideal_nr10_initexp7_d

5.5 Plain LoRA 对照记录

- eval_plain_control_bert_tiny_lora_sgd_t10000_e872_step1500_shuffle_lr3e3_r8_qv
- eval_plain_control_bert_tiny_lora_sgd_t10000_e872_step1500_shuffle_lr3e3_r8_qv_eval100_re